EE5203 L01 Lab Reference

The VS Code + STM32CubeMX workflow and first-day troubleshooting - Basri Erdogan

This page supports the L01 Lab Guide. It is a reference, not a second required manual.

The toolchain, by name

You use **two applications** and one invisible toolbox:

Tool	Job
STM32CubeMX	select the board, configure pins, generate the C project
VS Code (lab shortcut **VS Code - EE Labs**)	edit, build, flash, and debug that project
STM32CubeCLT (behind the scenes)	the compiler, CMake/Ninja build tools, ST-LINK debug server that VS Code drives

On laboratory PCs open VS Code through the **VS Code - EE Labs** desktop shortcut. Every extension the course needs is preinstalled and shared; you cannot add or remove extensions there, and you do not need to. The built-in terminal is Command Prompt.

Workflow reference

Stage	Where / action	Expected evidence
Create	STM32CubeMX → Board Selector → **NUCLEO-F401RE**; accept default initialization	board name in the project view
Configure	Pinout & Configuration tab	PA5 labeled LD2 / GPIO output
Generate	Project Manager: name (no spaces), location `C:\Work\<studentnumber>`, **Toolchain/IDE = CMake** → Generate Code	project folder with `Core/Inc`, `Core/Src`, `main.c`, `CMakeLists.txt`, `<name>.ioc`
Template	copy the **folder** `C:\Users\Public\EE5203_template\.vscode` into the generated project folder, next to the `.ioc` (once per project)	`<project>\.vscode` with five files: `tasks.json`, `launch.json`, `settings.json`, `fix_ld.cmake`, `STM32F401RETx_FLASH.ld`
Open	VS Code → File → Open Folder → the generated folder (the one that contains the `.ioc`). If CMake Tools asks for a preset choose **Debug**; if C/C++ offers to configure IntelliSense or create a build task, choose **No**	folder name in the title bar; `.vscode` visible in the Explorer
Edit	`Core/Src/main.c`, inside protected `USER CODE` regions	application statements preserved on regeneration
Build	**Ctrl+Shift+B** (Terminal → Run Build Task). Not the status-bar *Build* button: it skips the linker-script repair, see troubleshooting	`[N/N] Linking C executable <name>.elf` with no `FAILED` line; `.elf` in `build/Debug/`
Flash + run	press **F5** (or Run and Debug view → **Debug (ST-LINK)**). The in the editor's title bar is *not* the debugger	ST-LINK GDB server lines in the Debug Console; execution stops at `main`
Debug	debug toolbar + side panels	execution marker, breakpoints, variables

F5 is both the flash and the debug step — there is no separate "program the board" click in the normal loop. Press F5, then **Continue** to let the program run.

Debug controls (toolbar, left to right)

Control	Meaning
Continue (F5)	run until the next breakpoint
Pause	suspend a running program
Step over (F10)	execute the current line without entering called functions
Step into (F11)	enter the called function when debug info permits
Step out (Shift+F11)	run until the current function returns
Restart	reset and run again from the beginning
Stop (Shift+F5)	end the debug session

Evidence panels (debug side bar)

Panel	What it shows	Course name
VARIABLES / WATCH	C variables; add expressions to Watch	variable inspection
PERIPHERALS	live peripheral register bits (from the chip's SVD description)	**the SFR view** — register evidence lives here
MEMORY	raw memory bytes at an address	Memory view
Disassembly View (right-click → Open Disassembly View)	the machine instructions behind your C	Disassembly view

When a lab guide says *"inspect X in the SFR view"*, it means the **PERIPHERALS** panel.

Minimal application pattern

This is the layout CubeMX generates, with the lab's statements in place:

```
/* USER CODE BEGIN 2 */
int toggle_count = 0;
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
  HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
  toggle_count = toggle_count + 1;
  HAL_Delay(500);
}
/* USER CODE END 3 */
```

Two user regions touch the loop: `BEGIN WHILE … END WHILE` holds the loop opening, `BEGIN 3 … END 3` holds its body and closing brace. Put statements inside one of them — anything between `END WHILE` and `BEGIN 3` is outside both and disappears on regeneration from the `.ioc`.

First-day troubleshooting

Board not detected

- Confirm the board's power LED.

- Use the ST-LINK USB connector and a known **data** cable and port.
- Close other software that may hold ST-LINK (CubeMX programmer window, a second debug session).
- Reconnect the board, then press F5 again. The first F5 after plugging the board in sometimes fails on its own; a second F5 usually succeeds.

Build fails

- Save the edited file (unsaved dot on the tab?).
- Read the **first** relevant error in the terminal and its file/line.
- Check braces, parentheses, semicolons, and names.
- Confirm VS Code has the intended **folder** open (title bar).

F5 flashes but behavior is unchanged

- Confirm the build actually ran before the flash (watch the terminal).
- Set a breakpoint in the changed code — is it hit?
- Inspect `toggle_count` to determine whether the loop executes.

Linker error: `STM32F401RETx_FLASH.ld:56: syntax error`

- Known STM32CubeMX bug: the generated linker script loses its memory-region names (`ORIGIN() + LENGTH()`, `} >` with nothing after it).
- Build with **Ctrl+Shift+B**: the template's first task repairs the file before compiling. The status-bar *Build* button skips that step.
- Manual fix: copy `C:\Users\Public\EE5203_template\.vscode\STM32F401RETx_FLASH.ld` over the one in your project folder. CubeMX breaks it again on every regeneration, so keep building with Ctrl+Shift+B.

VS Code asks you to "choose a debugger", or F5 does nothing

- The `.vscode` folder is missing from the project, or you opened the wrong folder. Check that `<project>\.vscode\launch.jso` exists and that the title bar shows the project folder, then Ctrl+Shift+P → **Reload Window**.
- Use F5 or the next to **Debug (ST-LINK)** in the Run and Debug view, not the in the editor's title bar.

CubeMX offers to download the STM32CubeF4 package

- The package is already on the lab PC. Help → **Updater Settings** → Repository Folder: type `C:/Users/Public/STM32Cube/Repository` (do not browse into the package folder), OK, restart CubeMX.
- A notification in VS Code about installing "cube bundle(s)": answer **No**.

Code disappears after regeneration

- Move application changes into protected `USER CODE` regions.
- Treat the `.ioc` as the generated-configuration source of truth: pin changes happen in CubeMX, code changes in VS Code.

Working on lab PCs

- Log in with your own student account. Your folder `C:\Work\<studentnumber>` is created at login; generate every project there, never on the network drive and never on the Desktop.
- Project names: letters, digits and underscores only. No spaces, no Turkish characters. The build names its output after the folder.
- Every new project needs the `.vscode` template copied in once (Workflow reference, *Template* row). When you copy a whole project, the folder comes with it.
- End of session: run the desktop **Save to H** shortcut. It copies `C:\Work\<studentnumber>` (without `build/`) to `H:\EE_Labs`.
- Next session, possibly on another PC: copy the project back from `H:\EE_Labs` into `C:\Work\<studentnumber>` **before** opening it. Building and debugging directly on `H:` is not supported.

Help request format

Use this template:

```
Expected:
Observed:
Evidence collected:
Smallest next test:
```